

Numerical Solution of the

One-Dimensional Wave Equation

Using Implicit Finite Difference Methods

Course: Applied Mathematics / Numerical Analysis

Student Name: _____

Date: June 29, 2026

1. Introduction

This project presents a complete numerical study of the one-dimensional wave equation, a fundamental partial differential equation (PDE) that governs the propagation of disturbances through a continuous medium. The work implements an Implicit Finite Difference scheme in Python, covering simulation, energy analysis, convergence testing, and inverse wave-speed identification.

The wave equation arises naturally in acoustics (sound waves), structural mechanics (vibrating strings and beams), electrodynamics (electromagnetic waves), seismology, and quantum mechanics. Its mathematical richness makes it a canonical test case for numerical PDE solvers.

Objectives of this project:

- Formulate and implement an implicit finite difference scheme for the 1D wave equation.
- Derive and verify second-order spatial and temporal accuracy.
- Compute the mechanical energy of the vibrating string over time.
- Perform convergence analysis to confirm theoretical error rates.
- Solve an inverse problem to identify an unknown wave speed from a single observation.

The problem studied is a forced wave equation on the unit interval $[0, 1]$ with Dirichlet boundary conditions, whose exact solution is known analytically. This allows rigorous validation of the numerical scheme.

2. Mathematical Background

2.1 The One-Dimensional Wave Equation

The governing PDE is the second-order hyperbolic equation:

$$u_{tt}(x,t) - c^2 u_{xx}(x,t) = f(x,t), \quad \text{for } (x,t) \in (0,L) \times (0,T]$$

where:

- $u(x,t)$ is the transverse displacement of the string at position x and time t .
- $c > 0$ is the wave speed (determined by the physical properties of the string: $c = \sqrt{T/\rho}$, where T is tension and ρ is linear mass density).
- $f(x,t)$ is a forcing function (external load per unit mass).
- u_{tt} and u_{xx} denote the second partial derivatives with respect to time and space, respectively.

2.2 Initial and Boundary Conditions

To obtain a unique solution, the PDE is supplemented with:

Boundary Conditions (Dirichlet - pinned ends):

$$u(0,t) = 0, \quad u(L,t) = 0, \quad \text{for all } t \geq 0$$

Initial Conditions:

$$\begin{aligned} u(x,0) &= \sin(\pi x) && \text{(initial displacement)} \\ u_t(x,0) &= 0 && \text{(initial velocity is zero)} \end{aligned}$$

These conditions model a guitar string plucked into a half-sine shape and then released from rest.

2.3 Exact Solution (for Validation in Q3)

For the specific forcing function used in Question 3:

$$f(x,t) = -3\pi^2 \sin(\pi x) \cos(2\pi t)$$

the exact closed-form solution can be verified by substitution to be:

$$u_{\text{exact}}(x,t) = \sin(\pi x) \cos(2\pi t)$$

Verification: Computing $u_{tt} = -4\pi^2 \sin(\pi x) \cos(2\pi t)$ and $u_{xx} = -\pi^2 \sin(\pi x) \cos(2\pi t)$, we confirm that $u_{tt} - u_{xx} = (-4\pi^2 + \pi^2) \sin(\pi x) \cos(2\pi t) = -3\pi^2 \sin(\pi x) \cos(2\pi t) = f(x,t)$. The exact solution is a standing wave oscillating in time.

2.4 The Implicit Finite Difference Scheme

The continuous domain is discretized using a uniform grid:

$$\begin{aligned}x_i &= i \cdot \Delta x, & i &= 0, 1, \dots, N & \text{where } \Delta x &= L/N \\t_n &= n \cdot \Delta t, & n &= 0, 1, \dots, M & \text{where } \Delta t &= T/M\end{aligned}$$

The second-order central difference approximations for the derivatives are:

$$\begin{aligned}u_{tt} &\approx (u_i^{n+1} - 2u_i^n + u_i^{n-1}) / \Delta t^2 \\u_{xx} &\approx (u_{i+1}^n - 2u_i^n + u_{i-1}^n) / \Delta x^2\end{aligned}$$

Substituting into the PDE and rearranging for an implicit scheme (using a time-averaged spatial term) yields:

$$-r \cdot u_{i-1}^{n+1} + (1+2r) \cdot u_i^{n+1} - r \cdot u_{i+1}^{n+1} = 2u_i^n - u_i^{n-1} + \Delta t^2 f_i^n$$

where $r = (c\Delta t/\Delta x)^2$ is the squared Courant number. This is the key equation the code implements.

2.5 Matrix Formulation

Collecting the $(N-1)$ interior equations for $i = 1, \dots, N-1$ into a linear system $A \cdot u^{n+1} = b$, the matrix A is symmetric positive definite and tridiagonal:

$$A = \text{tridiag}(-r, \quad 1+2r, \quad -r) \quad (\text{size } (N-1) \times (N-1))$$

The right-hand side vector is: $b_i = 2u_i^n - u_i^{n-1} + \Delta t^2 f_i^n$.

2.6 Mechanical Energy

The total mechanical energy of the vibrating string at time t is defined as:

$$E(t) = (1/2) \int_0^L [u_t(x,t)^2 + c^2 u_x(x,t)^2] dx$$

The first term represents kinetic energy (due to velocity u_t) and the second represents potential/elastic energy (due to strain u_x). For the unforced wave equation, this energy is conserved exactly; numerical schemes may exhibit small drift due to discretization error.

2.7 Convergence Theory

The truncation error of the scheme is $O(\Delta t^2 + \Delta x^2)$, meaning the local error per step is proportional to the square of both grid spacings. When refining the grid, we expect the global error to decrease quadratically. Measured in the infinity norm (max error) or the discrete L^2 norm, we expect:

$$E_\infty \approx C \cdot h^2 \quad \text{and} \quad E_2 \approx C \cdot h^2 \quad \text{as } h = \Delta x \rightarrow 0$$

On a log-log plot, this appears as a straight line with slope 2.

3. Libraries Used

3.1 NumPy (numpy)

NumPy is Python's foundational library for numerical computation. It provides:

- `numpy.zeros`, `numpy.ones`, `numpy.linspace`: array initialization and grid creation.
- `numpy.diag`: construction of diagonal matrices.
- `numpy.gradient`: numerical differentiation using central finite differences.
- `numpy.abs`, `numpy.max`, `numpy.sum`, `numpy.sqrt`: vectorized mathematical operations for error norms.
- `numpy.sin`, `numpy.cos`, `numpy.pi`: evaluation of the exact solution and forcing function.

Why NumPy instead of plain Python lists? NumPy arrays store data in contiguous memory as C-style arrays, enabling highly optimized BLAS/LAPACK operations. A loop-based approach would be 100-1000x slower for the grid sizes used here.

3.2 SciPy (scipy)

SciPy extends NumPy with higher-level scientific algorithms. Three submodules are used:

scipy.linalg

- `scipy.linalg.lu(A)`: Computes the LU factorization $A = P \cdot L \cdot U$, where P is a permutation matrix, L is unit lower-triangular, and U is upper-triangular. This factorization is computed once and reused at every time step.
- `scipy.linalg.solve_triangular`: Solves triangular systems ($L \cdot y = b$ and $U \cdot x = y$) efficiently in $O(n^2)$ operations (compared to $O(n^3)$ for a general solver).

scipy.integrate

- `scipy.integrate.simpson`: Implements the Composite Simpson's Rule for numerical integration. Given a function evaluated at $N+1$ equally spaced points, it achieves $O(h^4)$ accuracy.

scipy.optimize

- `scipy.optimize.root_scalar`: A unified interface for scalar root-finding algorithms. Used here with the 'bisect' and 'newton' methods for the inverse wave speed problem.

3.3 Matplotlib

Matplotlib is Python's primary 2D plotting library. Components used:

- `pyplot.figure`, `pyplot.plot`, `pyplot.loglog`, `pyplot.show`: creating and displaying static plots.
- `pyplot.subplots`: creating figure/axes objects for animation.
- `FuncAnimation` (from `matplotlib.animation`): creates frame-by-frame animations.
`ani.save(..., writer='pillow')` exports the result as an animated GIF.

4. Code Explanation — Line by Line

4.1 Imports

```
import numpy as np
```

Imports the NumPy library. All array operations, trigonometric functions, and mathematical constants are accessed via the np namespace.

```
import scipy.linalg as la
```

Imports SciPy's linear algebra module. Used for LU factorization (la.lu) and triangular solvers (la.solve_triangular).

```
import scipy.integrate as integrate
```

Imports SciPy's integration module. Used for the Simpson's Rule integrator (integrate.simpson).

```
from scipy.optimize import root_scalar
```

Imports only the root_scalar function from scipy.optimize (selective import to keep the namespace clean). This function handles both bisection and Newton's method for the inverse problem.

```
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
```

Imports Matplotlib for visualization and FuncAnimation specifically for building the animated GIF of the vibrating string.

4.2 Function: solve_wave(N, M, L=1.0, T_end=2.0, c=1.0, is_q3=True)

This is the central numerical engine of the entire project. It implements the full implicit finite difference time-stepping algorithm.

Parameters:

Parameter	Default	Description
N	(required)	Number of spatial subintervals. Creates N+1 grid points $x_0, \dots, x_N$.
M	(required)	Number of time steps. Creates M+1 time levels $t_0, \dots, t_M$.
L	1.0	Length of the domain $[0, L]$.
T_end	2.0	Final simulation time.
c	1.0	Wave speed. Used in the Courant number $r = (c\Delta t/\Delta x)^2$.

is_q3	True	If True, uses the Q3 forcing function. If False, sets $f = 0$ (free vibration for Q5).
-------	------	--

Step 1: Grid Setup

```
dx = L / N
```

Computes the spatial step size $\Delta x = L/N$. The N subintervals partition $[0, L]$ into $N+1$ points.

```
dt = T_end / M
```

Computes the temporal step size $\Delta t = T_{\text{end}}/M$. Similarly partitions the time axis.

```
r = (c * dt / dx) ** 2
```

Computes the squared Courant number $r = (c\Delta t/\Delta x)^2$. This is the fundamental dimensionless parameter of the wave equation discretization. Unlike explicit schemes which require $r \leq 1$ (CFL condition) for stability, the implicit scheme is unconditionally stable for all $r > 0$, which is why it is preferred here.

```
x = np.linspace(0, L, N + 1)
```

Creates the spatial grid $[x_0, x_1, \dots, x_N]$ with $N+1$ equally spaced points.

```
t = np.linspace(0, T_end, M + 1)
```

Creates the temporal grid $[t_0, t_1, \dots, t_M]$ with $M+1$ equally spaced time levels.

Step 2: Solution Array and Forcing Function

```
U = np.zeros((N + 1, M + 1))
```

Allocates the solution matrix U of shape $(N+1) \times (M+1)$. Each column $U[:,n]$ stores the complete spatial profile $u(x_i, t_n)$ at time level n . Initializing to zero is correct since boundary values $u(0,t) = u(L,t) = 0$ remain zero throughout (automatically satisfied by only updating interior points $i=1,\dots,N-1$).

```
def f(x_val, t_val):
    if is_q3: return -3 * np.pi**2 * np.sin(np.pi * x_val) *
np.cos(2*np.pi*t_val)
    return np.zeros_like(x_val)
```

Defines the forcing function. When $\text{is_q3}=\text{True}$, this implements $f(x,t) = -3\pi^2 \sin(\pi x) \cos(2\pi t)$, which is the forcing that produces the exact solution $\sin(\pi x) \cos(2\pi t)$. When $\text{is_q3}=\text{False}$ (Question 5), $f = 0$ and the wave propagates freely. `np.zeros_like(x_val)` returns a zero array matching the shape and dtype of `x_val`, ensuring vectorized compatibility.

Step 3: Initial Conditions

```
U[:, 0] = np.sin(np.pi * x)
```

Sets the initial displacement: $u(x_i, 0) = \sin(\pi x_i)$ for all i . This is evaluated vectorially across all $N+1$ grid points simultaneously. Note that $\sin(\pi \cdot 0) = \sin(\pi \cdot 1) = 0$, so boundary conditions are automatically satisfied.

```
U[:, 1] = np.copy(U[:, 0])
```

Sets the second time level ($t_1 = \Delta t$). Because $u_t(x,0) = 0$ (the string is released from rest), the first-order Taylor approximation gives $u(x,\Delta t) \approx u(x,0) + \Delta t \cdot u_t(x,0) = u(x,0)$. Thus $U[:,1] = U[:,0]$. Using `np.copy` ensures a full independent copy of the array, avoiding aliasing issues.

Step 4: Tridiagonal Matrix Construction

```
main_diag = (1 + 2*r) * np.ones(N - 1)
```

Creates the main diagonal of A. Every interior equation contributes a $(1+2r)$ coefficient to the diagonal, arising from the implicit treatment of u^{n+1} .

```
off_diag = -r * np.ones(N - 2)
```

Creates the super- and sub-diagonal entries, each equal to $-r$. These arise from the coupling of neighboring spatial points through the u_{xx} term.

```
A = np.diag(main_diag) + np.diag(off_diag, k=1) + np.diag(off_diag, k=-1)
```

Assembles the full $(N-1) \times (N-1)$ tridiagonal matrix A by adding three diagonal matrices: the main diagonal ($k=0$), the superdiagonal ($k=+1$), and the subdiagonal ($k=-1$). This is a symmetric positive definite matrix, ensuring the linear system always has a unique solution.

Step 5: LU Decomposition (Pre-computation)

```
P, L_mat, U_mat = la.lu(A)
```

Computes the LU factorization of A. Since A is constant (does not change between time steps), this factorization only needs to be done ONCE. The matrix P is the permutation matrix for partial pivoting (ensuring numerical stability), L_mat is unit lower-triangular, and U_mat is upper-triangular. Storing this factorization reduces the cost of each time step from $O(n^3)$ (naive Gaussian elimination) to $O(n^2)$ (forward/back substitution), an enormous speedup for large N.

Step 6: Time-Stepping Loop

```
for n in range(1, M):
```

Iterates over time levels $n = 1, 2, \dots, M-1$. At each step, the solution at level $n+1$ is computed from levels n and $n-1$. The loop starts at $n=1$ (not $n=0$) because we need two initial time levels ($n=0$ and $n=1$) to start the three-level scheme.

```
    U_current = U[1:N, n]
    U_prev = U[1:N, n-1]
```

Extracts the interior values (indices 1 to $N-1$) at the current and previous time levels. The slice `[1:N]` excludes boundary points (indices 0 and N), which are fixed at zero and never updated.

```
    F_n = f(x[1:N], t[n])
```

Evaluates the forcing function at all interior spatial points at the current time level t_n . This is a vectorized call, computing all $N-1$ values simultaneously.

```
b = 2*U_current - U_prev + (dt**2) * F_n
```

Assembles the right-hand side vector **b**. This implements exactly:

$$b_i = 2u_i^n - u_i^{n-1} + \Delta t^2 f_i^n$$

This is derived by rearranging the finite difference equation. The term $2u_i^n - u_i^{n-1}$ encodes the history of the string's motion.

```
y = la.solve_triangular(L_mat, np.dot(P.T, b), lower=True)
```

Step 1 of the two-step LU solve: solves $L_mat \cdot y = P^T \cdot b$. First `np.dot(P.T, b)` applies the permutation to **b** (equivalent to $P^{-1}b = P^Tb$ since P is orthogonal). Then `solve_triangular` solves the lower-triangular system by forward substitution. The argument `lower=True` tells the solver to use forward substitution (vs. backward for upper-triangular).

```
U[1:N, n+1] = la.solve_triangular(U_mat, y, lower=False)
```

Step 2: solves $U_mat \cdot u^{n+1} = y$ by back substitution (`lower=False` indicates upper-triangular). The result is stored directly into the $(n+1)$ -th column of **U**, at interior points only. Boundary values $U[0,n+1]$ and $U[N,n+1]$ remain zero (set during initialization).

4.3 Return Values of `solve_wave`

```
return x, t, U, dx, dt
```

Returns the spatial grid **x** (array of length $N+1$), temporal grid **t** (array of length $M+1$), solution matrix **U** (shape $(N+1) \times (M+1)$), and step sizes **dx** and **dt**. All subsequent analysis functions receive these arrays.

4.4 Function: `compute_energy(U, dx, dt, c)`

Computes the total mechanical energy $E(t)$ at each time step using numerical differentiation and integration.

```
u_x = np.gradient(U, dx, axis=0)
```

Computes the spatial derivative $\partial u / \partial x$ at every grid point and every time level using central finite differences. `np.gradient` uses second-order accurate central differences for interior points and first-order differences at boundaries. The argument `dx=dx` specifies the spacing for correct scaling.

```
u_t = np.gradient(U, dt, axis=1)
```

Computes the temporal derivative $\partial u / \partial t$ at every grid point and every time level. `axis=1` specifies differentiation along the time axis (columns of **U**).

```
energy = np.zeros(U.shape[1])
```

Initializes the energy array with $M+1$ zeros, one entry per time level.

```
for n in range(U.shape[1]):
    integrand = u_t[:,n]**2 + (c**2) * u_x[:,n]**2
    energy[n] = 0.5 * integrate.simpson(integrand, dx=dx)
```

For each time level n , assembles the integrand $u_t^2 + c^2 \cdot u_x^2$ at all spatial points, then applies the Composite Simpson's Rule to approximate the integral over $[0, L]$. The factor 0.5

comes from the definition $E = (1/2) \int [\dots] dx$. Simpson's Rule achieves $O(\Delta x^4)$ accuracy (two orders higher than the trapezoidal rule), making it appropriate here.

5. Execution Script — Question by Question

5.1 Question 3: Simulation, Visualization, and Convergence

```
N_test, M_test = 50, 100
```

Sets up a moderate grid for initial testing: 50 spatial intervals and 100 time steps. The ratio $M/N = 2$ keeps the Courant number $r = (1 \cdot (T/M)/(\Delta x))^2$ moderate and the scheme stable.

```
x, t, U, dx, dt = solve_wave(N_test, M_test, is_q3=True)
```

Runs the primary simulation with the Q3 forcing. The five returned values are unpacked directly.

```
exact_final = np.sin(np.pi*x) * np.cos(2*np.pi*2.0)
```

Evaluates the exact solution $u(x, 2.0) = \sin(\pi x) \cdot \cos(4\pi)$ at all grid points. Note that $\cos(4\pi) = 1$, so the exact solution at $T=2$ is simply $\sin(\pi x)$.

Static Plot (Q3.2 & Q3.4):

```
times_to_plot = [0, int(M_test/4), int(M_test/2), M_test]
```

Selects four representative time indices: $t=0$, $t=0.5$, $t=1.0$, and $t=2.0$. Plotting at these snapshots reveals the oscillatory pattern: at $t=0$ and $t=2$, the wave should match the initial condition (since $\cos(2\pi \cdot t) = 1$ at $t=0$ and $t=2$); at $t=0.5$ and $t=1.5$, $\cos(\pi) = -1$ and the wave is inverted; at $t=0.25$ and $t=0.75$, $\cos(\pi/2) = 0$ and the string momentarily flattens.

Animation (Q3.3):

```
ani = FuncAnimation(fig, update, frames=range(0, M_test, 2), blit=True)
```

Creates a frame-by-frame animation, using every other time step (step 2) to reduce the file size. The update function receives the frame index and updates the plotted y-data with $U[:, \text{frame}]$. The `blit=True` option redraws only changed artists, improving rendering speed.

```
ani.save('wave_animation.gif', writer='pillow', fps=30)
```

Saves the animation as a GIF using the Pillow image library at 30 frames per second. Pillow is a pure-Python image library and does not require external video codecs.

Convergence Analysis (Q3.5):

```
N_values = [10, 20, 40, 80, 160]
```

Defines a sequence of grid resolutions, each doubling the previous. Doubling N (and setting $M = 2N$) halves Δx and Δt simultaneously, allowing clean observation of the $O(h^2)$ convergence rate.

```
E_inf = np.max(np.abs(exact_res - num_final))
```

Computes the maximum absolute error (infinity norm / sup norm) between the exact and numerical solutions at the final time $T=2.0$. This measures the worst-case error at any grid point.

```
E_2 = np.sqrt(dx_val * np.sum((exact_res - num_final)**2))
```

Computes the discrete L^2 norm of the error. The factor $\sqrt{\Delta x}$ normalizes the sum to approximate the continuous integral $\int |e(x)| dx$, making E_2 independent of grid resolution.

```
plt.loglog(dx_list, E_inf_list, 's-', ...)
```

Plots both errors on a log-log scale. A second-order method appears as a straight line with slope 2. If the slope is exactly 2, it confirms the theoretical prediction $E \approx C \cdot h^2$.

5.2 Question 4: Energy Analysis

```
energy = compute_energy(U, dx, dt, c=1.0)
```

Calls the energy function using the solution from Q3. The result is an array of $M+1$ energy values.

For the forced problem in Q3, the forcing function injects or removes energy, so $E(t)$ is not conserved. The plot should reveal whether energy oscillates or drifts over time, which can be compared to the analytical energy of the exact solution: $E_{\text{exact}} = (1/2) \int [(2\pi \sin(\pi x) \sin(2\pi t))^2 + (\pi \cos(\pi x) \cos(2\pi t))^2] dx = \pi^2/4 \times (4\sin^2(2\pi t) + \cos^2(2\pi t))$.

5.3 Question 5: Inverse Wave Speed Identification

This section formulates and solves a nonlinear inverse problem: given a single observation of the displacement at a fixed point and time, recover the unknown wave speed c .

Problem Setup:

```
t_star = 0.5
u_obs = 0.35
```

The observation is $u(L/2, 0.5) = 0.35$. We seek c such that the simulation reproduces this value.

Residual Function $F(c)$:

```
def F(c_guess):
    x_g, t_g, U_g, _, _ = solve_wave(20, 40, c=c_guess, is_q3=False)
    mid_x_idx = len(x_g) // 2
    t_star_idx = int((t_star / 2.0) * 40)
    return U_g[mid_x_idx, t_star_idx] - u_obs
```

$F(c)$ is defined as the difference between the simulated value at the midpoint and the observation. A lower resolution grid ($N=20$, $M=40$) is used to reduce the computational cost of the many function evaluations needed by the root-finding algorithms. The time index $t_star_idx = \text{int}((0.5/2.0) * 40) = 10$ correctly maps $t=0.5$ to column index 10 out of 40 steps (since $\Delta t = 2.0/40$).

Bisection Method (Q5.1):

```
result_bisect = root_scalar(F, bracket=[0.1, 2.0], method='bisect',
                             xtol=1e-3)
```

The bisection method requires a bracket $[a, b]$ where $F(a)$ and $F(b)$ have opposite signs. It repeatedly halves the interval, discarding the half where F doesn't change sign, converging

to the root at a guaranteed linear rate. The tolerance $\text{xtol}=1\text{e-}3$ means the root is located to within 0.001. The result serves as the starting point for Newton's method.

Newton's Method (Q5.2):

```
fprime_approx = lambda c_val: (F(c_val + 0.001) - F(c_val)) / 0.001
result_newton = root_scalar(F, x0=c_approx, fprime=fprime_approx,
method='newton')
```

Newton's method uses the derivative $F'(c)$ to construct a linear approximation of F at each iteration: $c_{\{k+1\}} = c_k - F(c_k)/F'(c_k)$. It converges quadratically (the number of correct digits roughly doubles each iteration), far faster than bisection. Since $F'(c)$ is not available analytically (F is a numerical function), it is approximated by a forward finite difference with step 0.001. Starting from the bisection estimate ensures convergence to the correct root.

6. Link Between Code and Mathematics

This section explicitly maps each mathematical formula to its code implementation.

Mathematical Object	Code Implementation	Notes
$\Delta x = L/N$	<code>dx = L / N</code>	Spatial resolution
$\Delta t = T/M$	<code>dt = T_end / M</code>	Temporal resolution
$r = (c\Delta t/\Delta x)^2$	<code>r = (c*dt/dx)**2</code>	Squared Courant number
$u(x,0) = \sin(\pi x)$	<code>U[:,0] = np.sin(np.pi*x)</code>	Initial displacement
$u_t(x,0) = 0$	<code>U[:,1] = np.copy(U[:,0])</code>	Zero initial velocity
$A = \text{tridiag}(-r, 1+2r, -r)$	<code>np.diag(main)+np.diag(off,1)+np.diag(off,-1)</code>	System matrix
$b_i = 2u_i^n - u_i^{n-1} + \Delta t^2 f_i^n$	<code>b = 2*U_current - U_prev + dt**2*F_n</code>	RHS vector
$A = \text{PLU}$ (factorization)	<code>P, L_mat, U_mat = la.lu(A)</code>	Pre-computed once
$Ly = P^T b,$ $Uu^{n+1} = y$	<code>solve_triangular</code> (twice)	Forward/back substitution
$E = (1/2) \int [u_t^2 + c^2 u_x^2] dx$	<code>0.5*simpson(integrand, dx=dx)</code>	Simpson's Rule quadrature
$F(c) = u(L/2, t^*; c) - u_{\text{obs}}$	<code>def F(c_guess): ... return U_g[mid,t_idx]-u_obs</code>	Root-finding residual

7. Algorithm Explanation

7.1 Overall Flow

The algorithm processes as follows:

- INPUT: Grid parameters N , M , physical parameters L , T_{end} , c , problem flag is_q3.
- GRID SETUP: Create spatial grid $x[i]$ and temporal grid $t[n]$; compute Δx , Δt , r .
- INITIALIZATION: Set $U[:,0] = \sin(\pi x)$ and $U[:,1] = U[:,0]$ (zero initial velocity).
- MATRIX ASSEMBLY: Build tridiagonal matrix $A = \text{tridiag}(-r, 1+2r, -r)$.
- LU FACTORIZATION: Compute P , L , $U = \text{lu}(A)$ once before the time loop.
- TIME LOOP ($n = 1$ to $M-1$): Evaluate forcing f at t_n ; assemble RHS b ; solve $Au^{n+1} = b$ via two triangular solves; store result in $U[:,n+1]$.
- OUTPUT: Return x , t , U (full solution matrix), dx , dt .

The energy computation loops over all time levels, computing spatial/temporal derivatives via `np.gradient` and integrating via Simpson's Rule.

The inverse problem wraps the entire PDE solver in a scalar function $F(c)$ and applies bisection followed by Newton's method to find the root.

8. Numerical Analysis Discussion

8.1 Stability

The key advantage of the implicit scheme is unconditional stability. The explicit ("leapfrog" or FTCS) scheme for the wave equation requires the Courant-Friedrichs-Lewy (CFL) condition $c \cdot \Delta t / \Delta x \leq 1$ to be satisfied, severely limiting the allowable time step. The implicit scheme avoids this restriction entirely: the matrix $A = \text{tridiag}(-r, 1+2r, -r)$ is strictly diagonally dominant for all $r > 0$, guaranteeing that all eigenvalues of A lie in the right half of the complex plane, ensuring stability.

8.2 Accuracy

The truncation error is $O(\Delta t^2 + \Delta x^2)$. Both the time and space discretizations use second-order central differences, so refining the grid by a factor of 2 reduces the error by approximately a factor of 4. The convergence analysis in Q3.5 verifies this empirically.

8.3 Computational Complexity

Without LU pre-computation, solving $Au^{n+1} = b$ at each of M time steps would cost $O(M \cdot n^3)$ for a general $n \times n$ matrix. However:

- The LU factorization is performed once: $O(n^3)$ cost.
- Each time step requires two triangular solves: $O(n^2)$ each.
- Total cost: $O(n^3 + M \cdot n^2)$.
- For a tridiagonal system, the LU factorization can be done in $O(n)$ and each solve in $O(n)$, giving total $O(M \cdot n)$. The code uses the general `la.lu` rather than a tridiagonal-specific solver, leaving room for optimization.

8.4 Memory Usage

The solution matrix U has $(N+1) \times (M+1)$ entries. For $N=M=100$, this is $101 \times 101 \approx 10,000$ doubles = ~80 KB. For $N=M=1000$, this grows to ~8 MB. For very large simulations, it would be more memory-efficient to only store two time levels (current and previous), but the full matrix is kept here for visualization.

8.5 Error in Energy Computation

The `np.gradient` function uses first-order differences at the boundaries and second-order at interior points, introducing small boundary errors. Simpson's Rule provides $O(\Delta x^4)$ accuracy. The dominant error in $E(t)$ comes from the $O(\Delta t^2 + \Delta x^2)$ error in U itself, propagated through

the derivatives. For the unforced problem, any observed drift in energy is a measure of the numerical dissipation of the scheme.

9. Results and Interpretation

9.1 Wave Displacement Plots (Q3.2 & Q3.4)

The static snapshots at $t = 0, 0.5, 1.0, 2.0$ should reveal a standing wave. At $t=0$, the wave matches $\sin(\pi x)$. At $t=0.5$, $\cos(\pi) = -1$, so the wave is inverted to $-\sin(\pi x)$. At $t=1.0$, $\cos(2\pi) = 1$, returning to the original. The numerical solution at $T=2.0$ should match the exact solution to within machine precision $\times (N, M)$, typically $O(10^{-4})$ for moderate grids.

9.2 Animation (Q3.3)

The GIF should show a sinusoidal wave oscillating up and down (a pure standing wave). The amplitude varies as $\cos(2\pi t)$, reaching maximum at integer t , zero crossing at $t = 0.25, 0.75, 1.25, 1.75$, and minimum at $t = 0.5, 1.5$. No traveling wave components are present because the initial condition and forcing both involve only the first spatial mode.

9.3 Convergence Analysis (Q3.5)

On the log-log plot, both E_∞ and E_2 should appear as straight lines with slope approximately 2. A line with slope 2 means that halving Δx reduces the error by a factor of 4. If the slope is slightly less than 2, it may be due to the lower-order boundary treatment in `np.gradient` or the use of the Q3 forcing function evaluation at a fixed time rather than averaging. Deviations from slope 2 at coarse grids are normal; asymptotic convergence is typically only visible at fine grids.

9.4 Energy Plot (Q4)

For the forced problem, the energy should oscillate in time. The total energy at time t of the exact solution is $E_{\text{exact}}(t) = (\pi^2/4)(4\sin^2(2\pi t) + \cos^2(2\pi t))$, which is a non-constant function. The numerically computed energy should closely track this. Any difference reflects the $O(\Delta x^2)$ accuracy of the spatial derivative approximation.

9.5 Inverse Problem (Q5)

The bisection method should converge in approximately $\log_2 ((2.0-0.1)/0.001) \approx 11$ iterations to isolate the root within the tolerance. Newton's method then refines this to machine precision in 3-5 additional iterations, demonstrating its quadratic convergence. The final value of c should be such that the simulated displacement at the midpoint at $t=0.5$ equals 0.35.

10. Conclusion

This project has demonstrated a complete end-to-end workflow for the numerical study of the one-dimensional wave equation. The key accomplishments are:

- Implementation of an unconditionally stable implicit finite difference scheme that solves the PDE at each time step by solving a tridiagonal linear system via pre-computed LU decomposition, achieving significant computational efficiency.
- Verification of second-order spatial and temporal accuracy through a systematic convergence analysis on five grid levels, confirming the theoretical $O(h^2)$ rate on both the infinity norm and the discrete L^2 norm.
- Computation of the total mechanical energy $E(t)$ using Composite Simpson's Rule on numerically differentiated fields, showing the dynamic energy exchange in the forced string system.
- Solution of a nonlinear inverse problem (wave speed identification) using a hybrid Bisection–Newton approach, combining the robustness of bisection for bracket refinement with the quadratic convergence of Newton's method for high-accuracy refinement.

Advantages of the chosen implementation:

- Unconditional stability allows large time steps without CFL restriction, important for long-time simulations.
- LU pre-computation amortizes the factorization cost over all M time steps.
- Vectorized NumPy operations throughout eliminate explicit Python loops wherever possible, maximizing performance.
- Modular design: `solve_wave`, `compute_energy`, and the root-finding code are independent and reusable.

Possible improvements:

- Use a tridiagonal-specific solver (e.g., `scipy.linalg.solve_banded` or the Thomas algorithm) to reduce the per-step cost from $O(n^2)$ to $O(n)$.
- Store only two time levels of U instead of the full matrix to reduce memory usage for large grids.
- Use Richardson extrapolation to accelerate convergence beyond second order.
- Add a higher-order initial condition for $U[:,1]$ by including the second-order term $(dt^2/2) \cdot u_{tt}(x,0)$.
- For the inverse problem, replace the finite-difference derivative approximation with automatic differentiation (e.g., JAX) for exact gradients.

This implementation stands as a rigorous, well-documented foundation for studying hyperbolic PDEs numerically, balancing mathematical correctness, computational efficiency, and pedagogical clarity.

End of Report